

25S CST8288 Lab 013

High-Level Design

Public Transit Fleet Management System (PTFMS)

Solution Overview and Improvement Guidance

Version 2.0

By Annabel Cheng

Submitted to Teddy Yap, Gustavo Adami

in partial fulfillment of the requirements for CST8288 Final Project

Computer Programming

Algonquin College

August 08, 2025

Declaration Of Sole Authorship

I, Annabel Cheng, confirm that this work submitted for assessment is my own and is expressed in my own words. Any uses made within it of the works of any other author, in any form (ideas, equations, figures, texts, tables, programs), are properly acknowledged at the point of use. A list of the references is included.

GitHub Repo:

<https://github.com/c-annabel/PublicTransitFleetManagementSystem.git>

Digital Signatures:

Annabel Cheng

Date: August 08, 2025

Abstract

This report presents the design, implementation, and evaluation of the Public Transit Fleet Management System (PTFMS), a web-based application developed to streamline the management of public transit operations. The system integrates core functionalities including user authentication, vehicle management, GPS tracking, fuel and energy monitoring, predictive maintenance scheduling, and performance reporting. Built on a three-tier architecture and implemented in Java with MySQL as the database backend, PTFMS applies established design patterns—such as DAO, Builder, Command, Strategy, Observer, and Simple Factory—to ensure modularity, scalability, and maintainability. The report details the system's architecture, data models, security considerations, testing strategies, and deployment approach, supported by UML diagrams for both structural and behavioural views. Recommendations for future enhancements, including real-time data integration, advanced analytics, and cloud-based deployment, are also provided to guide the continued evolution of the platform.

Table of Contents

LIST OF FIGURES	5
LIST OF TABLES.....	5
GLOSSARY	6
1. INTRODUCTION	7
2. TARGETED AUDIENCE	7
3. SCOPE	7
3.1 IN SCOPE	7
3.2 OUT-OF-SCOPE	8
4. APPLICATION ARCHITECTURE	8
4.1 PRESENTATION LAYER	8
4.2 BUSINESS LAYER	9
4.3 DATA LAYER	9
4.4 FILE STRUCTURE DIAGRAM	9
5. BUSINESS ARCHITECTURE	9
5.1 ACTORS	9
5.2 USE CASE DIAGRAM	11
5.3 USE CASES	12
5.4 RELATIONSHIP	12
6. DETAILED DESIGN.....	12
6.1 COMPONENT VIEW (3-TIER REALISATION)	13
6.2 CLASS DESIGN & PATTERN REALISATION	14
6.2.1 FR01 – User Registration & Authentication (DAO)	14
6.2.2 FR02 – Vehicle Management (Builder + DAO).....	15
6.2.3 FR03 – GPS Tracking & Operator Breaks (Command + DAO).....	16
6.2.4 FR04 – Fuel/Energy Monitoring (Strategy + Observer + DAO).....	17
6.2.5 FR05 – Predictive Maintenance (Command + DAO)	19
6.2.6 FR06 – Reporting & Analytics (Simple Factory + DAO).....	20
7. DATA ARCHITECTURE	22
7.1 LOGICAL DATA MODEL (ERD).....	22
7.2 PHYSICAL DATA MODEL (SCHEMA DIAGRAM)	23
7.3 SUMMARY	25
8. SECURITY ARCHITECTURE	25
8.1 AUTHENTICATION & AUTHORIZATION	25
8.2 DATA SECURITY AND SESSION MANAGEMENT.....	25
9. DEPLOYMENT ARCHITECTURE.....	26
9.1 INFRASTRUCTURE	26

9.2	DEPLOYMENT MODEL	26
10.	TESTING MODEL	27
10.1	FUNCTIONAL TESTING APPROACH	27
10.2	AUTOMATED UNIT AND INTEGRATING TESTING	27
10.3	TESTING SUMMARY	28
11.	CONCLUSION	28
	REFERENCES	28

List of Figures

Figure 1: Use Case Diagram	11
Figure 2 PTFMS Component Diagram	13
Figure 3: FR01 – DAO UML Class Diagram.....	14
Figure 4: FR01 Login Sequence Diagram.....	15
Figure 5: FR02 - Builder UML Class Diagram	15
Figure 6: FR02 Add Vehicle Sequence Diagram	16
Figure 7: FR03 Command UML Class Diagram.....	17
Figure 8: FR03 Starting Break Sequence Diagram.....	17
Figure 9: FR04 Strategy UML Class Diagram.....	18
Figure 10: FR04 Observer UML Class diagram.....	18
Figure 11: FR04 Consumption Check & Alert Sequence Diagram	19
Figure 12: FR05 Command UML Class Diagram	20
Figure 13: FR05 Booking Maintenance Sequence Diagram	20
Figure 14: FR06 Simple Factory UML Class Diagram	21
Figure 15: FR06 Generate Report Sequence Diagram	21
Figure 16: Logical Entity–Relationship Diagram (ERD)	22
Figure 17 Physical Data Model (Schema Diagram)	24

List of Tables

Table 1 PTFMS File Structure	Error! Bookmark not defined.
------------------------------------	-------------------------------------

Glossary

- GoF** The "Gang of Four" are the four authors of the highly influential 1994 book, "Design Patterns: Elements of Reusable Object-Oriented Software."
- Object-Oriented design** A software development approach that models a system as a collection of interacting objects, each with its own state and behaviors.
- UML** Unified Modeling Language, a standardized graphical notation used to model and document the design of a software system.
- Sequence Diagram** A type of UML diagram that shows the interactions between objects in a chronological order, illustrating the sequence of messages exchanged to perform a specific task.
- ERD** Entity Relationship Diagram is a visual representation of the database's logical structure, showing entities, their attributes, and the relationships between them to guide efficient data organization and integrity in the system.

1. Introduction

This report provides a high-level design overview of the Public Transit Fleet Management System (PTFMS). The PTFMS is a web-based application designed to assist public transit agencies in monitoring, tracking, and optimizing their fleet operations, which include buses, electric light rail, and diesel-electric trains. By integrating real-time GPS tracking, fuel/energy monitoring, and predictive maintenance alerts, this solution aims to enhance the overall efficiency and reliability of public transit services. The high-level design outlined here serves as a foundational blueprint, detailing the application's architecture, key components, and intended functionality to guide the subsequent stages of development and refinement.

2. Targeted Audience

The development team and transit management stakeholders are the intended audiences for this high-level design document. For the development team, it serves as the primary technical guide, providing a clear understanding of the proposed 3-tier architecture, the required design patterns (e.g., DAO, Builder), and the technology stack (Java 21, MySQL, Apache Tomcat). For transit management, the document offers a non-technical overview of how the system will meet the specified functional requirements, such as real-time vehicle tracking, fuel consumption reports, and predictive maintenance alerts. The content is structured to ensure both groups can grasp the project's scope, goals, and foundational design principles, facilitating effective collaboration and communication throughout the project lifecycle.

3. Scope

The scope of this high-level design document and the subsequent project implementation is strictly defined by the provided project requirements.

3.1 In Scope

The project develops the Public Transit Fleet Management System, a web-based application. A 3-tier architecture with GoF Design Patterns [1] like DAO, Builder, and four others from a list must be used. The scope includes the following fundamental functional requirements:

- **User registration and authentication:** Transit Managers and Operators can register and log in with role-based access to system functions.
- **Vehicle Management:** Transit Managers can register buses and trains with unique identifiers and fuel kinds.
- **GPS tracking:** Transit Managers can see station arrival and departure times using real-time vehicle position data. Operators can manually log out-of-service times.

- **Monitoring Energy/Fuel Consumption:** The system will generate fuel and energy usage reports for each vehicle type and alert Transit Managers about excessive consumption.
- **Predictive Maintenance:** The technology monitors key components and engine diagnostics to send Transit Managers maintenance notifications to schedule maintenance.
- **Reporting and Analytics:** Transit maintenance, operator performance, and cost dashboards are required.

The final deliverables include a high-level design document, UML diagrams, a MySQL database schema with example data, and a NetBeans Maven project containing HTML, Java code, and JUnit tests. Javadoc comments for public aspects must accompany any Java code.

3.2 Out-of-Scope

Any features or deliverables not specified in the project requirements are out of scope. Examples include:

- **Mobile Application Development:** The project is web-based and does not need mobile interfaces or functions.
- **Public-Facing Interface:** The system's users are restricted to Transit Managers and Operators, so a public-facing portal for passengers is out of scope.
- **Complex deployment:** The project uses Apache Tomcat as the server, but it does not require a production-level deployment plan, CI pipelines, or complex server-side setups.
- **Automated Data Entry:** The project mentions automobiles transmitting real-time GPS data, but the human logging of operator out-of-service intervals shows that a completely automated data entry system is not feasible for all use cases.

4. Application Architecture

The Public Transit Fleet Management System (PTFMS) is designed using a 3-tier architecture, which provides a clear separation of concerns to ensure the system is maintainable, scalable, and modular. This structure (See Table 1 PTFMS File Structure) divides the application into three distinct layers—Presentation, Business, and Data—with a central controller class managing the flow of data and requests between them. The project's file structure serves as a concrete implementation of this high-level design.

4.1 Presentation Layer

This layer is responsible for all user interaction and the display of the application's user interface. It is primarily implemented using Java Servlets and JavaServer Pages (JSPs). The presentation package contains the servlets (e.g., LoginServlet.java, ReportServlet.java) that handle user requests, while the webapp directory houses the JSPs (e.g., dashboard.jsp, reports.jsp) that render the graphical interface for both Transit Managers and Operators. The AuthFilter.java class within this layer is critical for enforcing role-based access control.

4.2 Business Layer

The Business Layer contains the core application logic and business rules. It is implemented within the business package and its sub-packages, where several GoF Design Patterns are applied to structure the logic. The business package includes various services (e.g., `UserService.java`, `VehicleService.java`) that encapsulate key business functionalities. The architecture incorporates the Observer pattern within the observer package to handle event-driven alerts. The Strategy pattern, found in the strategy package, provides a flexible way to calculate fuel and energy consumption for different vehicle types (e.g., `BusConsumptionStrategy.java`, `ElectricRailConsumptionStrategy.java`). The Command pattern is used in the command package to encapsulate requests as objects, and the `ReportFactory.java` class is used for generating various reports.

4.3 Data Layer

The Data Layer is responsible for the persistent storage and retrieval of all application data, which is managed by a MySQL database. This layer is implemented using the Data Access Object (DAO) pattern, as demonstrated by the *dataaccess* package. This package contains dedicated DAO classes for each entity (e.g., `UserDAO.java`, `VehicleDAO.java`) that abstract the database access logic. Data is transferred between the layers using Plain Old Java Objects (POJOs), as defined in the *transferobjects* package (e.g., `User.java`, `Vehicle.java`).

4.4 File Structure Diagram

This diagram (Table 1PTFMS File Structure) is a table listing each file name organized by Presentation Layer, Business Layer and Data Layer.

5. Business Architecture

This section outlines the functional requirements of the PTFMS from a user's perspective, using use case diagrams (*Figure 1: Use Case Diagram*) to illustrate the system's core capabilities. The data entities and their relationships, which are defined by these use cases, are further detailed in the Entity-Relationship Diagram (ERD) within the Data Architecture section.

5.1 Actors

The system involves two primary actors:

- **Manager:** The manager is responsible for overseeing the overall system, including viewing various reports and managing certain data.
- **Operator:** The operator interacts with the system to perform core operational tasks, such as logging maintenance and fueling records.

PTFMS File Structure

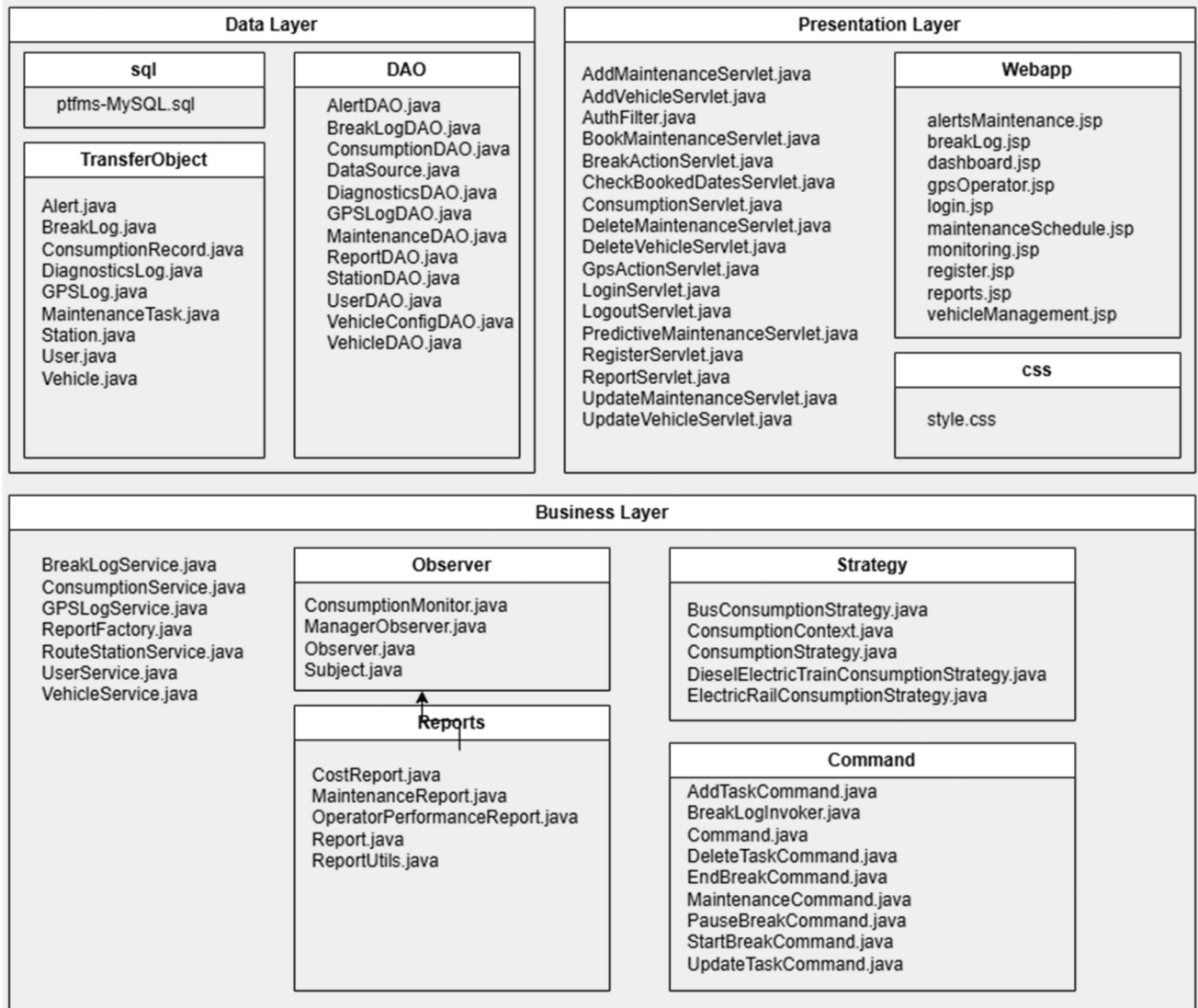


Table 1PTFMS File Structure

5.2 Use Case Diagram

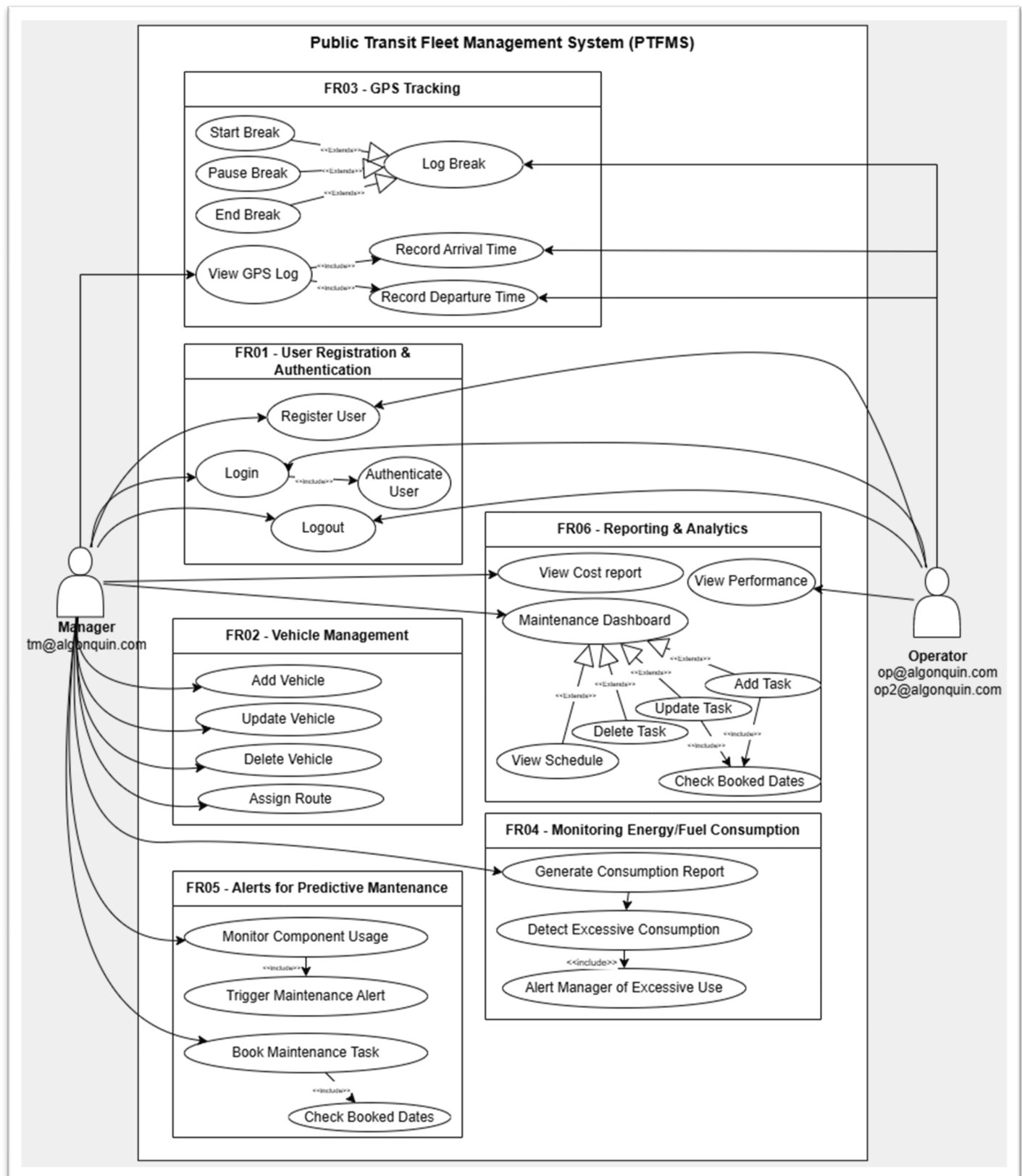


Figure 1: Use Case Diagram

5.3 Use Cases

The diagram outlines several core functionalities, or use cases, which are grouped by the project's functional requirements.

Manager-Specific Use Cases:

- **Vehicle Management (FR-02):** The manager can add, update, and delete vehicle records, as well as assign them to a route.
- **Reporting & Analytics (FR-06):** This is a central use case for the manager. It includes generating various reports such as <<include>> Maintenance Reports and <<include>> Cost Analysis Reports. A key part of this functionality is also viewing the Maintenance Dashboard.
- **Alerts for Predictive Maintenance (FR-05):** The manager is responsible for monitoring component usage, and based on triggered alerts, they can Book Maintenance Tasks.

Operator-Specific Use Cases:

- **User Registration & Authentication (FR-01):** Both **managers and operators can Register and Login to the system.**
- **GPS Tracking (FR-03):** The operator can Log Breaks and view their GPS log, which <<include>> recording arrival and departure times.
- **Performance report:** This Use Case allows the operator to see how they have performed in terms of on-time arrival rates and working efficiency.

5.4 Relationship

The relationships between the actors and use cases are clearly defined:

- The **Manager** is directly associated with use cases for vehicle management, reporting, and receiving consumption and maintenance alerts. They are specifically responsible for booking and monitoring maintenance tasks.
- The **Operator** is directly associated with use cases for logging their activities (breaks, GPS) and viewing their own performance.
- The **Generate Reports** use case <<include>>s Maintenance Reports and Cost Analysis Reports, indicating that these two reports are mandatory components of the main "Generate Reports" process.

6. Detailed Design

The Detailed Design describes how PTFMS is organised into a three-tier architecture, with components in the presentation, business, and data access layers working together to process requests and manage data. It also maps each functional requirement to its implemented design patterns, showing the classes, interfaces, and collaborations that realise the system's features in alignment with the chosen architecture.

6.1 Component View (3-Tier Realisation)

PTFMS is structured as a classic three-tier web system (See Figure 2 PTFMS Component Diagram). The **Presentation layer** comprises JSPs and servlets that expose FR01–FR06 use cases; the **Business layer** encapsulates domain logic, pattern implementations, and orchestration; the **Data Access layer** isolates persistence behind DAOs. Transfer Objects bridge layers without leaking JDBC concerns.

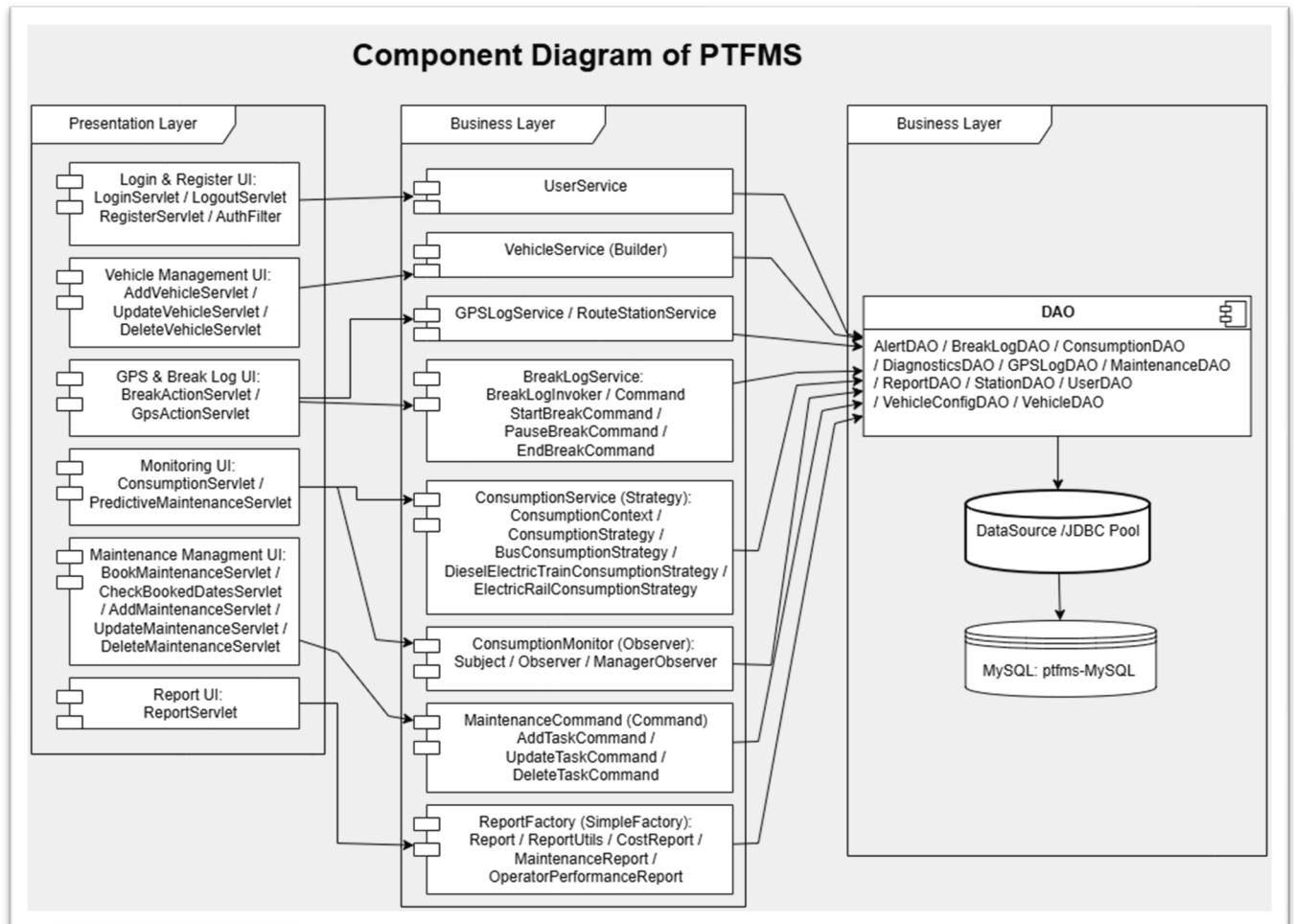


Figure 2 PTFMS Component Diagram

Presentation (Servlets/JSPs)

- **FR01 – Auth:** LoginServlet, RegisterServlet, LogoutServlet, AuthFilter (+ login.jsp, register.jsp, dashboard.jsp).
- **FR02 – Vehicle Mgmt:** AddVehicleServlet, UpdateVehicleServlet, DeleteVehicleServlet, vehicleManagement.jsp.
- **FR03 – GPS & Breaks:** GpsActionServlet, BreakActionServlet, gpsOperator.jsp, breakLog.jsp.
- **FR04 – Monitoring:** ConsumptionServlet, monitoring.jsp.
- **FR05 – Predictive Maintenance:** PredictiveMaintenanceServlet, maintenance JSPs (alertsMaintenance.jsp, maintenanceSchedule.jsp).
- **FR06 – Reports:** ReportServlet, reports.jsp.

Business (Services, Patterns, Orchestrators)

- Core services: UserService, VehicleService, GPSLogService, BreakLogService, ConsumptionService, RouteStationService.
- Reporting API: ReportFactory + reports package (MaintenanceReport, CostReport, OperatorPerformanceReport, ReportUtils).
- Patterns (see §7.2) implemented under command, strategy, and observer packages.

Data Access (DAOs + DataSource)

- DAOs: UserDao, VehicleDAO, StationDAO, RouteStationService (uses DAOs), GPSLogDAO, BreakLogDAO, ConsumptionDAO, DiagnosticsDAO, VehicleConfigDAO, MaintenanceDAO, ReportDAO, AlertDAO.
- Connection management: DataSource (singleton).
- **Transfer Objects:** User, Vehicle, Station, GPSLog, BreakLog, ConsumptionRecord, DiagnosticsLog, MaintenanceTask, Alert.
- This componentisation keeps servlets thin, centralises rules in services/patterns, and contains all SQL in DAOs for testability and substitution.
- Each decorator may augment the result as the response propagates back to the client.

6.2 Class Design & Pattern Realisation

DAO is the baseline pattern across FR01–FR06: every persistence interaction is mediated by the corresponding *DAO, called from services, never directly from servlets.

6.2.1 FR01 – User Registration & Authentication (DAO)

- **Flow:** LoginServlet/RegisterServlet → UserService → UserDao → User (TO).
- **Concern:** credential validation, role assignment, session control via AuthFilter.
- **UML class diagram** (Figure 3:FR01- DAO UML Class Diagram):

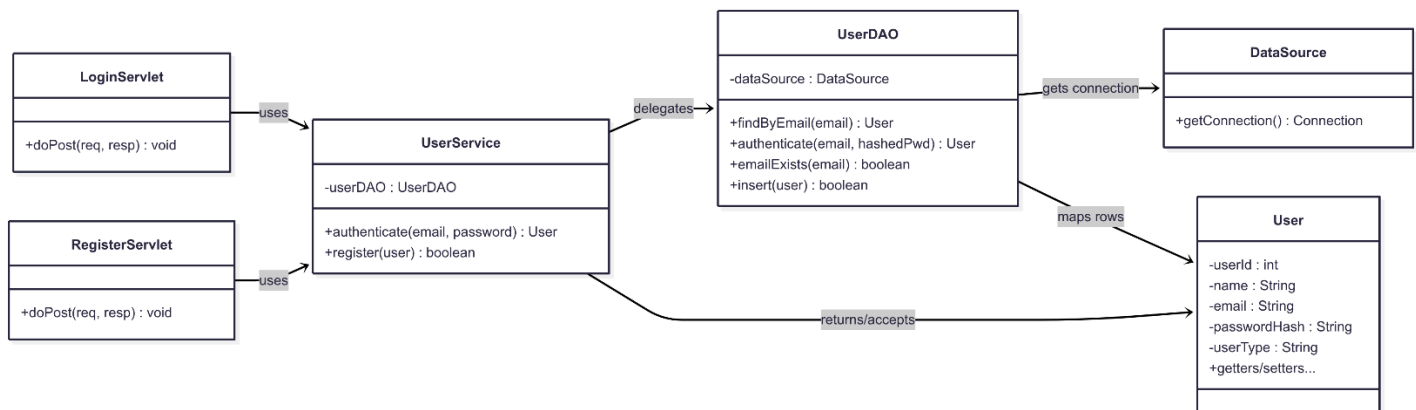


Figure 3: FR01 – DAO UML Class Diagram

- **Sequence Diagram for Login (Figure 3: FR01 Login Sequence Diagram):**

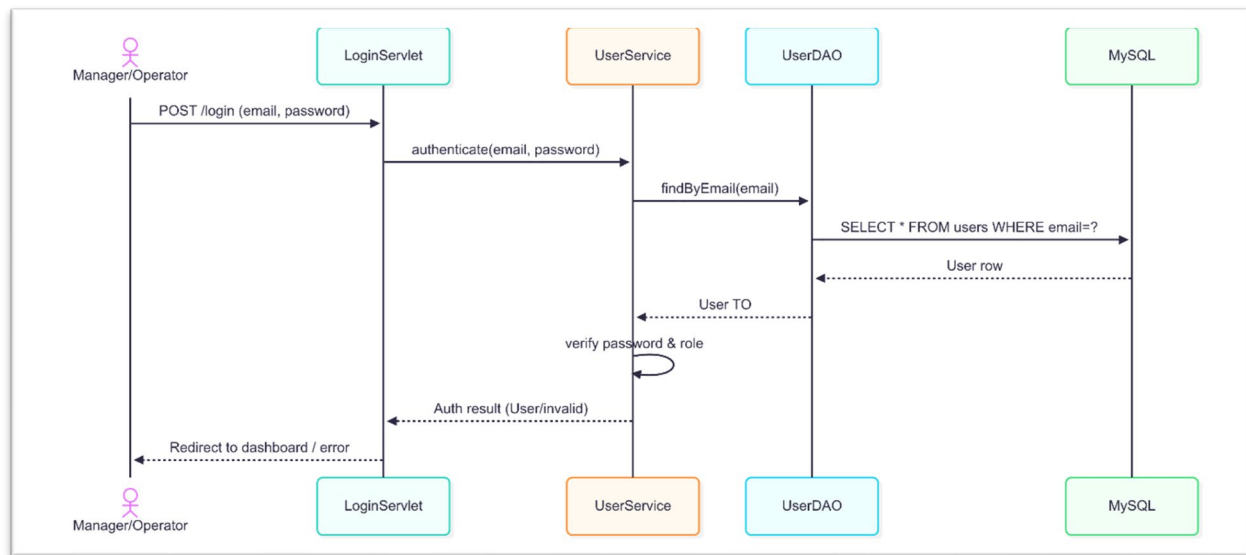


Figure 4: FR01 Login Sequence Diagram

6.2.2 FR02 – Vehicle Management (Builder + DAO)

- **Flow:** Add/Update/DeleteVehicleServlet → VehicleService → VehicleDAO → Vehicle (TO).
- **Pattern:** **Builder** for Vehicle object construction (multi-attribute creation without telescoping constructors).
- **UML Class Diagram:** *Vehicle Builder UML* (Director: VehicleService if used, Builder within Vehicle or separate VehicleBuilder), plus *Component diagram* showing servlet→service→DAO. (Figure 5: FR02 - Builder UML Class Diagram)

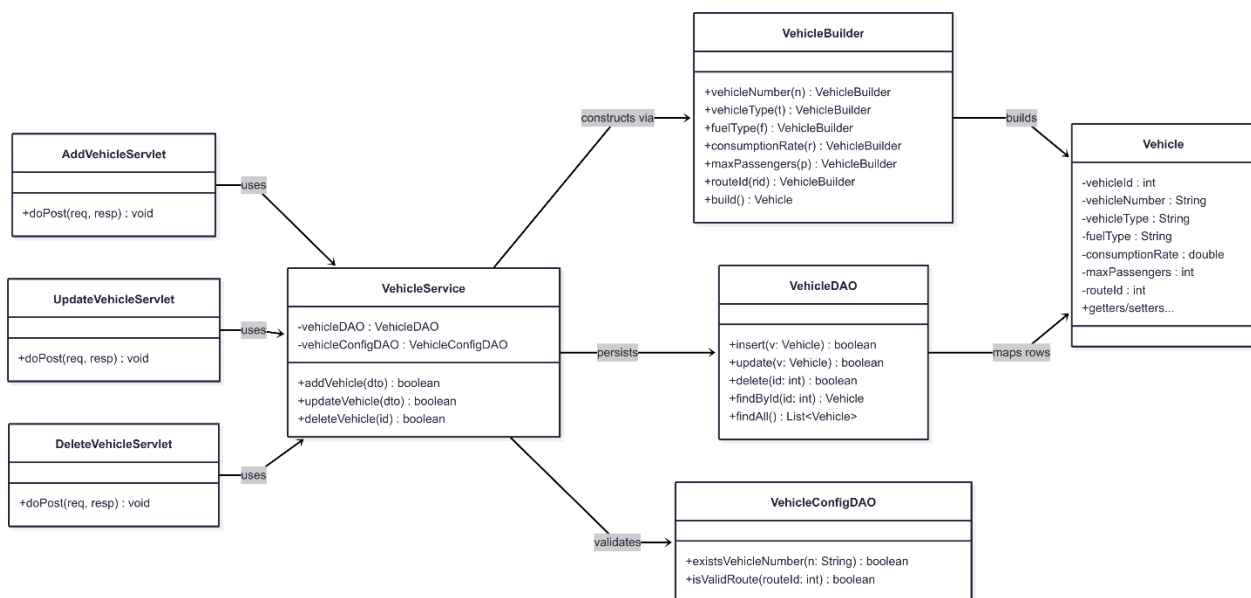


Figure 5: FR02 - Builder UML Class Diagram

- **Sequence Diagram for Adding Vehicle (Figure 4: FR02 Add Vehicle Sequence Diagram):**

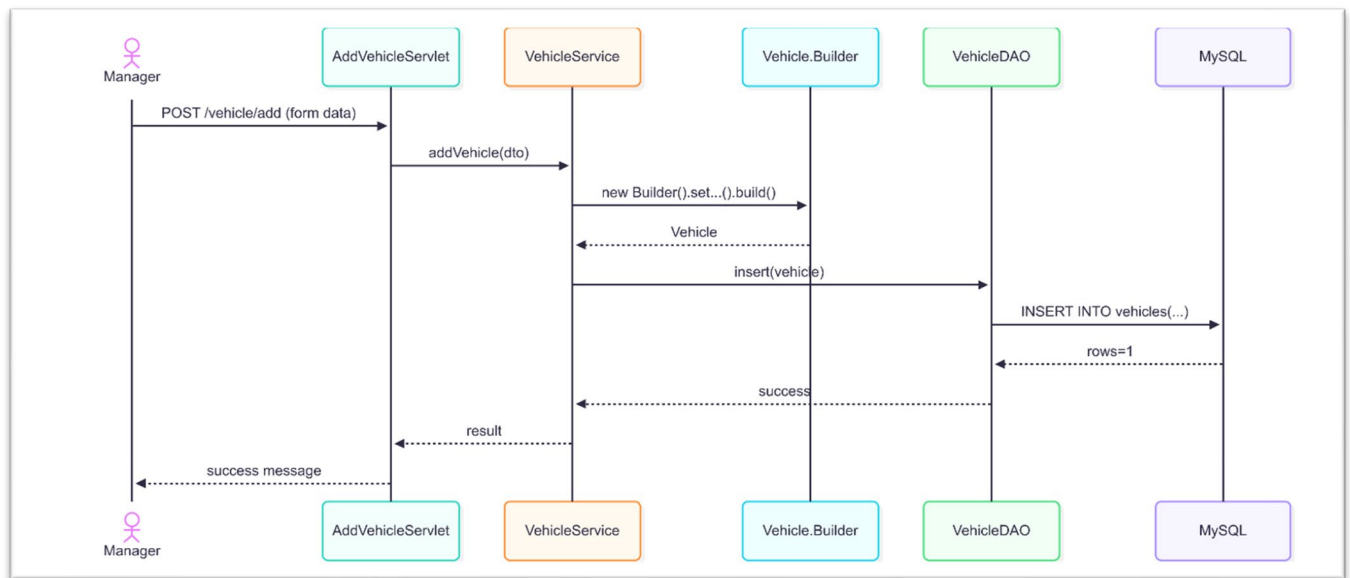


Figure 6: FR02 Add Vehicle Sequence Diagram

6.2.3 FR03 – GPS Tracking & Operator Breaks (Command + DAO)

- **GPS logs:** GpsActionServlet → GPSLogService → GPSLogDAO → GPSLog (TO).
- **Break actions (Command):**
 - Interface: Command
 - Concrete: StartBreakCommand, PauseBreakCommand, EndBreakCommand
 - Invoker: BreakLogInvoker
 - Context: BreakActionServlet → invoker → commands → BreakLogDAO → BreakLog (TO).
- **UML Class Diagrams:** Command pattern class diagram (Invoker–Command–ConcreteCommand–Receiver(DAO/Service)); Sequence for “Start Break”. (Figure 7: FR03 Command UML Class Diagram)

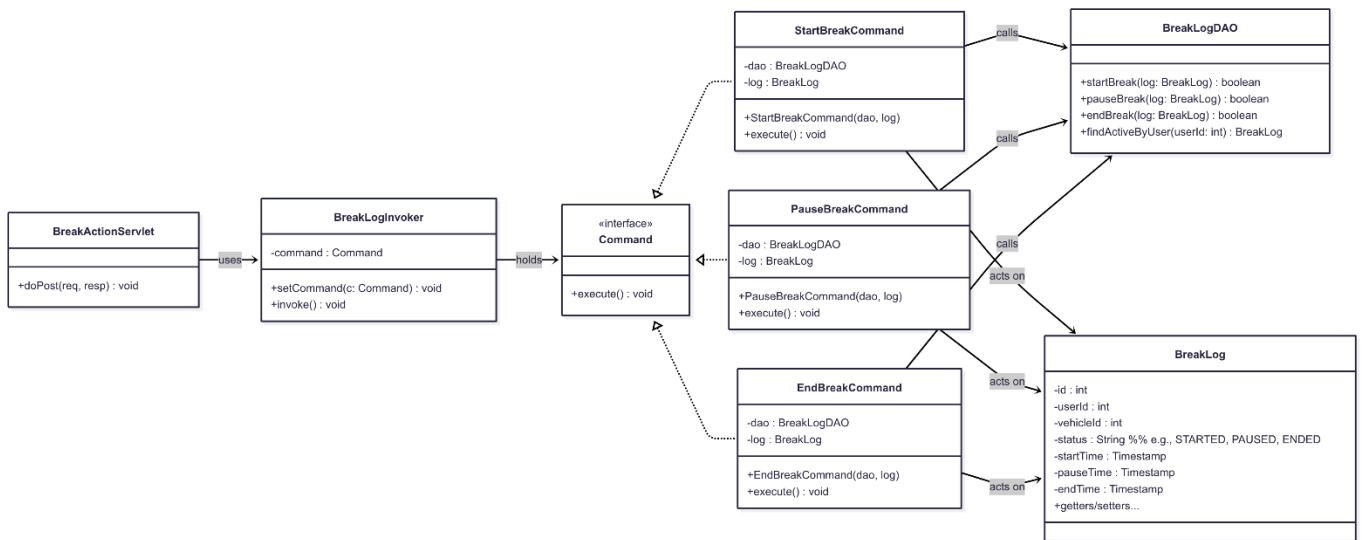


Figure 7: FR03 Command UML Class Diagram

- **Sequence Diagram for Starting (Figure 5: FR03 Starting Break Sequence Diagram):**

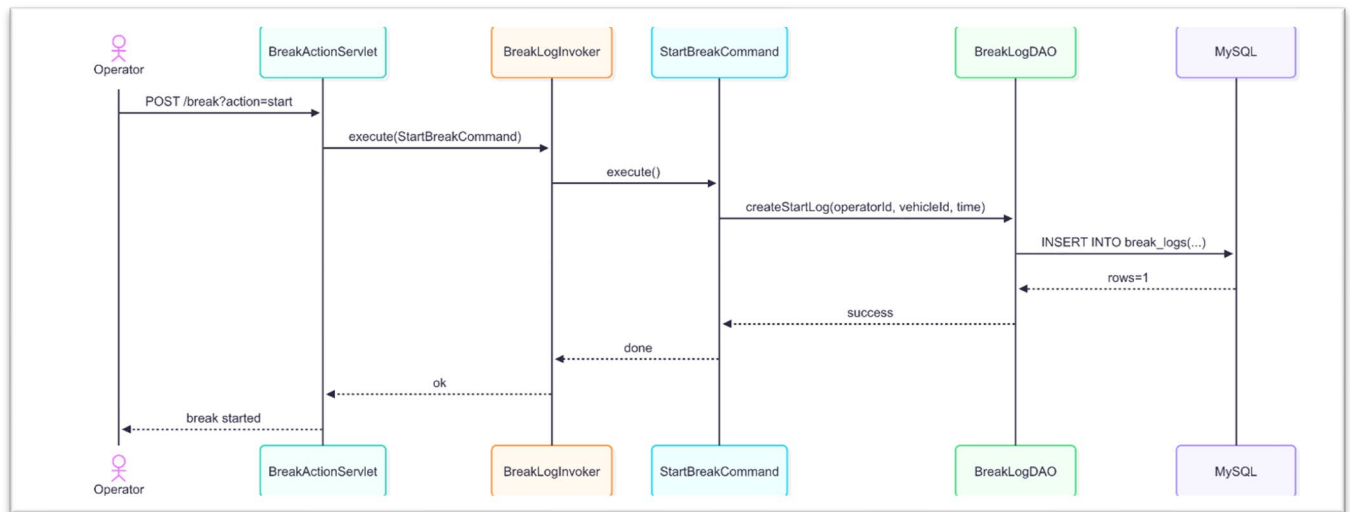


Figure 8: FR03 Starting Break Sequence Diagram

6.2.4 FR04 – Fuel/Energy Monitoring (Strategy + Observer + DAO)

- **Strategy:**
 - Context: ConsumptionContext (selected by ConsumptionService).
 - Interface: ConsumptionStrategy.
 - Concrete: BusConsumptionStrategy, ElectricRailConsumptionStrategy, DieselElectricTrainConsumptionStrategy.
- **Observer (threshold alerts):**
 - Subject: ConsumptionMonitor
 - Observer: ManagerObserver (notified when computed metrics exceed thresholds).

- **Flow:** ConsumptionServlet → ConsumptionService → ConsumptionDAO (data) → ConsumptionContext (compute) → ConsumptionMonitor (notify) → AlertDAO/UI.
- **UML Class Diagrams:** *Strategy class diagram* (Context–Strategy–Concretes) (Figure 9: FR04 Strategy UML Class Diagram) and *Observer class diagram* (Subject–Observer) (Figure 10: FR04 Observer UML Class diagram).

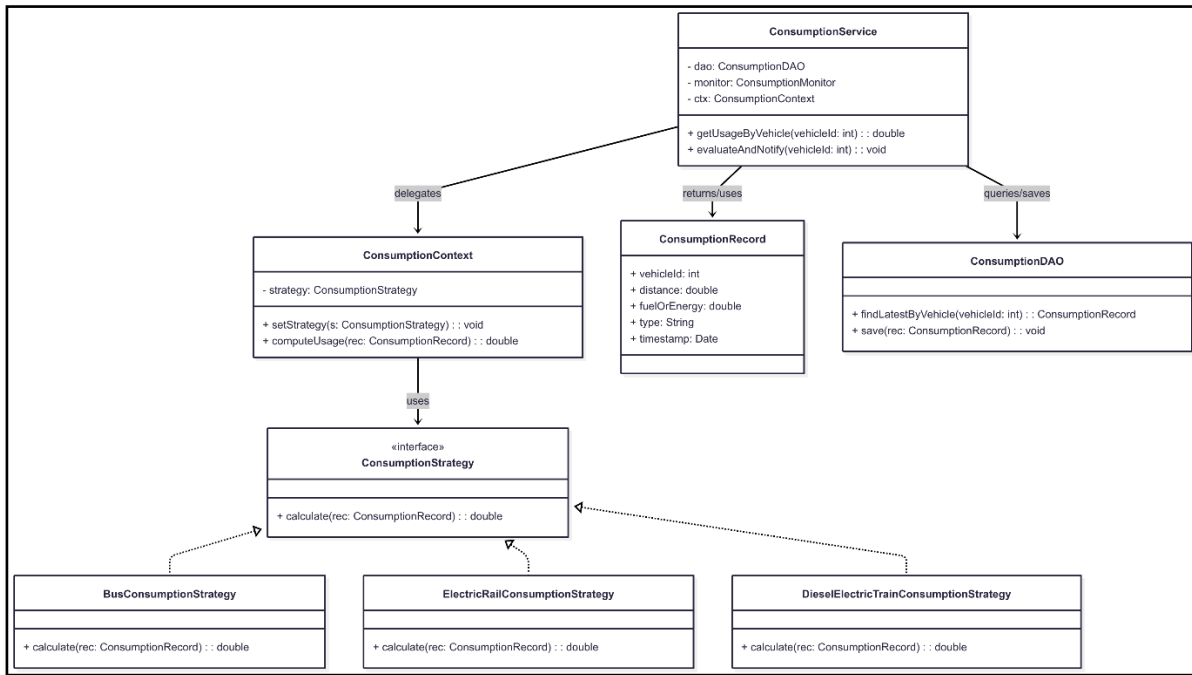


Figure 9: FR04 Strategy UML Class Diagram

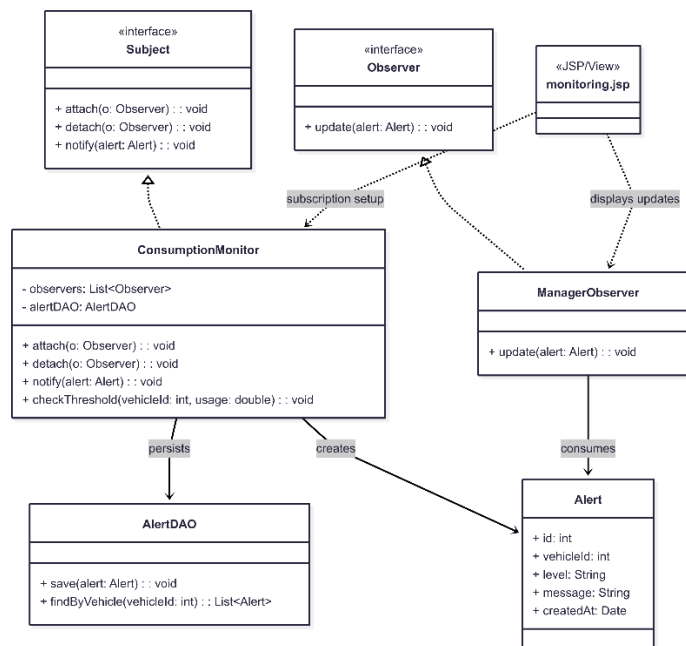


Figure 10: FR04 Observer UML Class diagram

- **Sequence Diagram for Consumption Check & Alert** (Figure 6: FR04 Consumption Check & Alert Sequence Diagram):

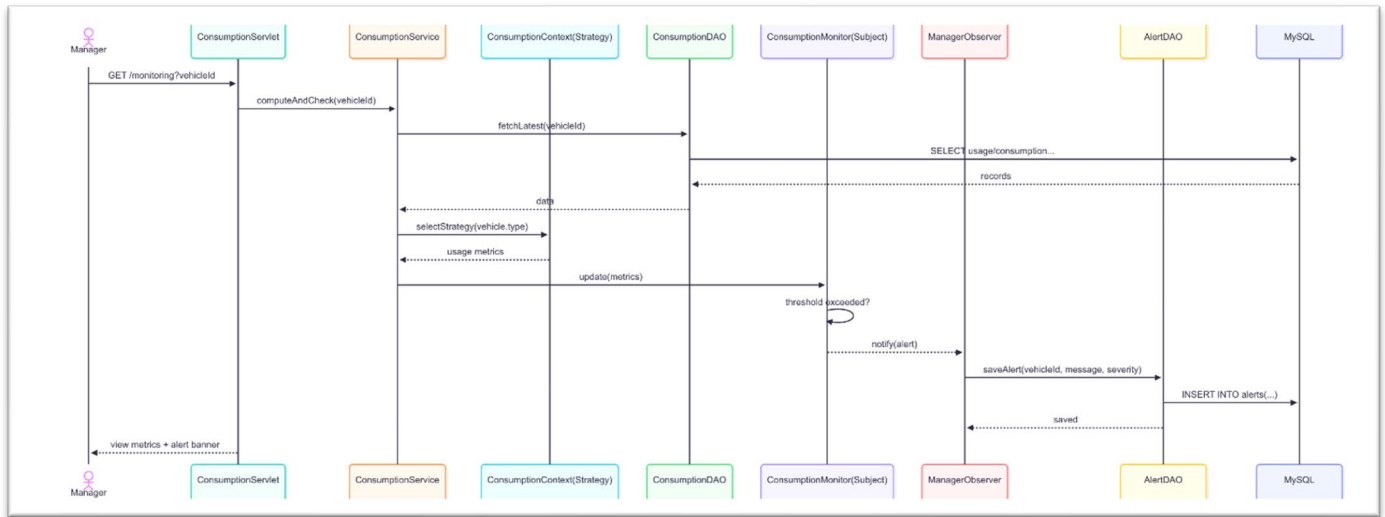


Figure 11: FR04 Consumption Check & Alert Sequence Diagram

6.2.5 FR05 – Predictive Maintenance (Command + DAO)

- **Commands for scheduling lifecycle:**
 - Interface: MaintenanceCommand.
 - Concrete: AddTaskCommand, UpdateTaskCommand, DeleteTaskCommand.
- **Flow:** Add/Update/DeleteMaintenanceServlet → command execution → MaintenanceDAO → MaintenanceTask (TO).
- **Rules:** date validation, uniqueness of scheduled date per shop-day policy, completed-task immutability checks handled in command/service.
- **Diagram:** Maintenance Command UML and Sequence for “Update Task with validation”. (Figure 12: FR05 Command UML Class Diagram)

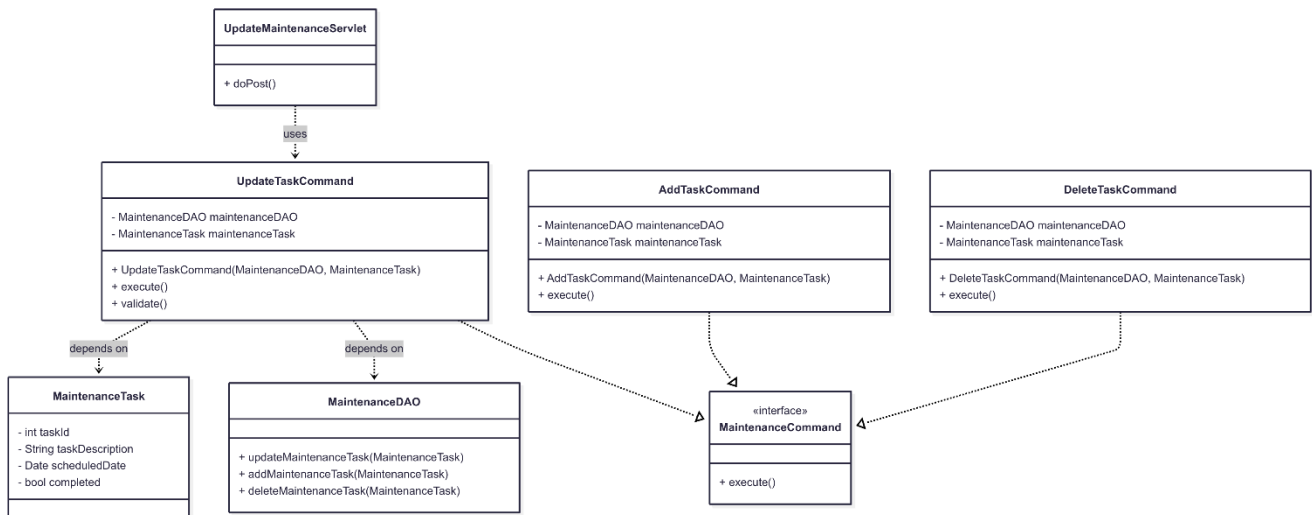


Figure 12: FR05 Command UML Class Diagram

- **Sequence Diagram for Booking Maintenance Task (Figure 7: FR05 Booking Maintenance Sequence Diagram):**

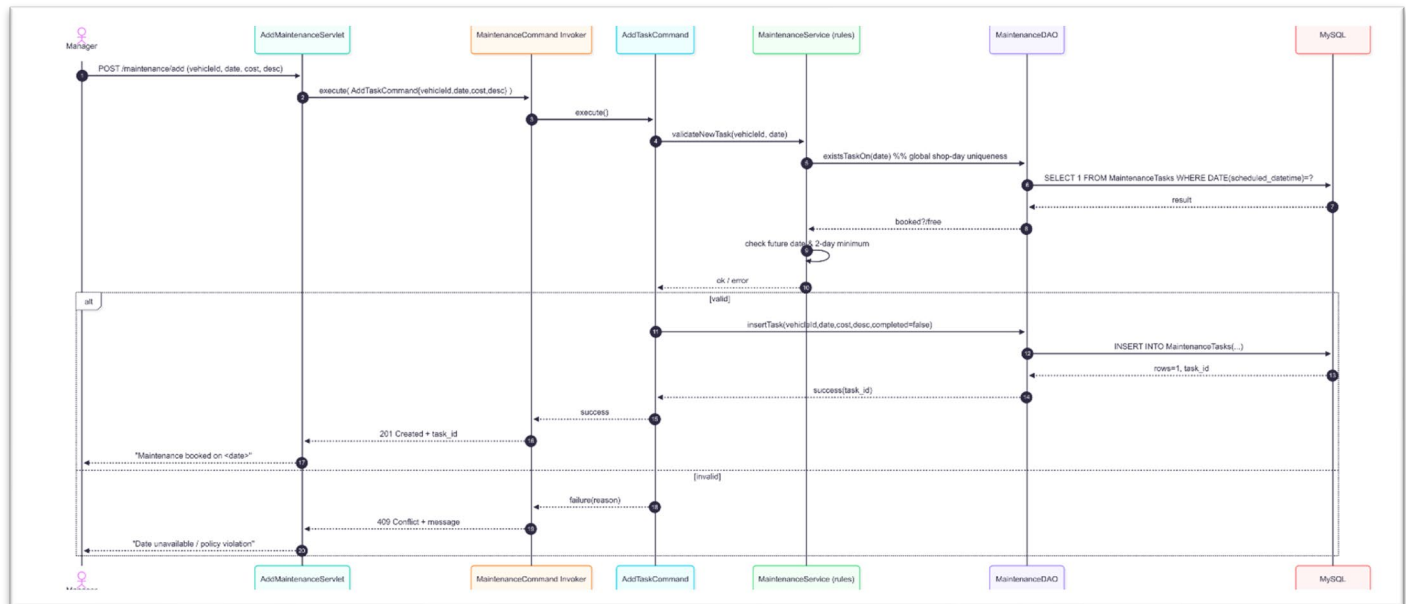


Figure 13: FR05 Booking Maintenance Sequence Diagram

6.2.6 FR06 – Reporting & Analytics (Simple Factory + DAO)

- **Factory:** ReportFactory → returns MaintenanceReport / CostReport / OperatorPerformanceReport implementing common Report interface; helper ReportUtils.
- **Flow:** ReportServlet → ReportFactory → selected Report → DAOs (ReportDAO, ConsumptionDAO, MaintenanceDAO, etc.).
- **Diagram:** Simple Factory class diagram plus Component diagram showing report selection. (Figure 14: FR06 Simple Factory UML Class Diagram)

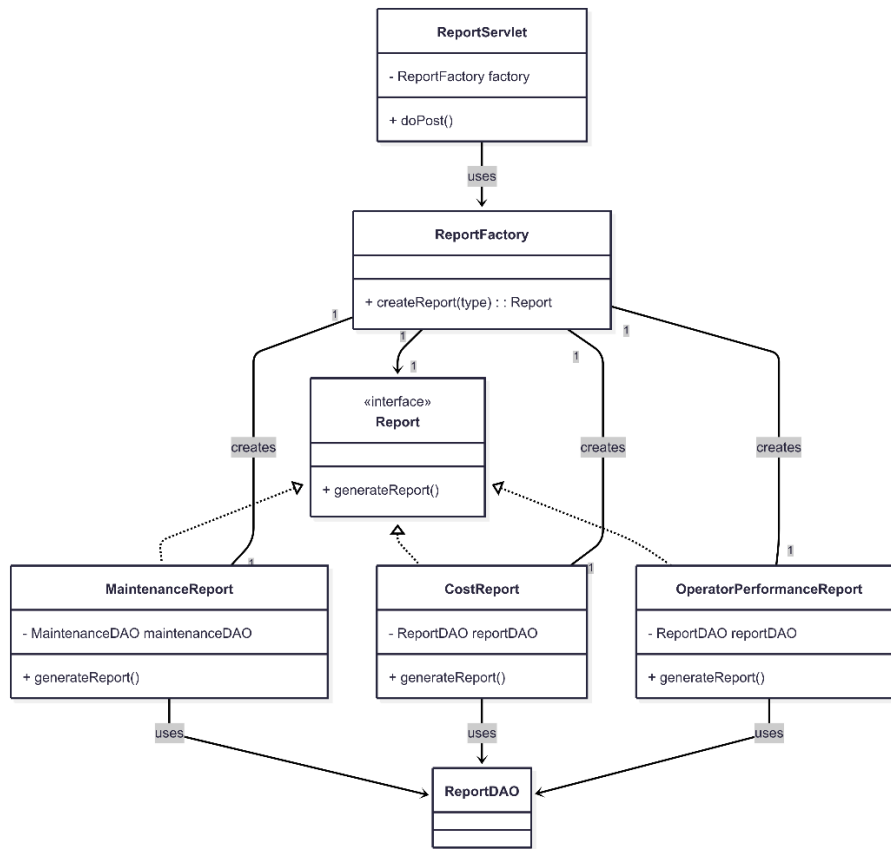


Figure 14: FR06 Simple Factory UML Class Diagram

- **Sequence Diagram for Generating Reports** (Figure 7: FR06 Generate Report Sequence Diagram):

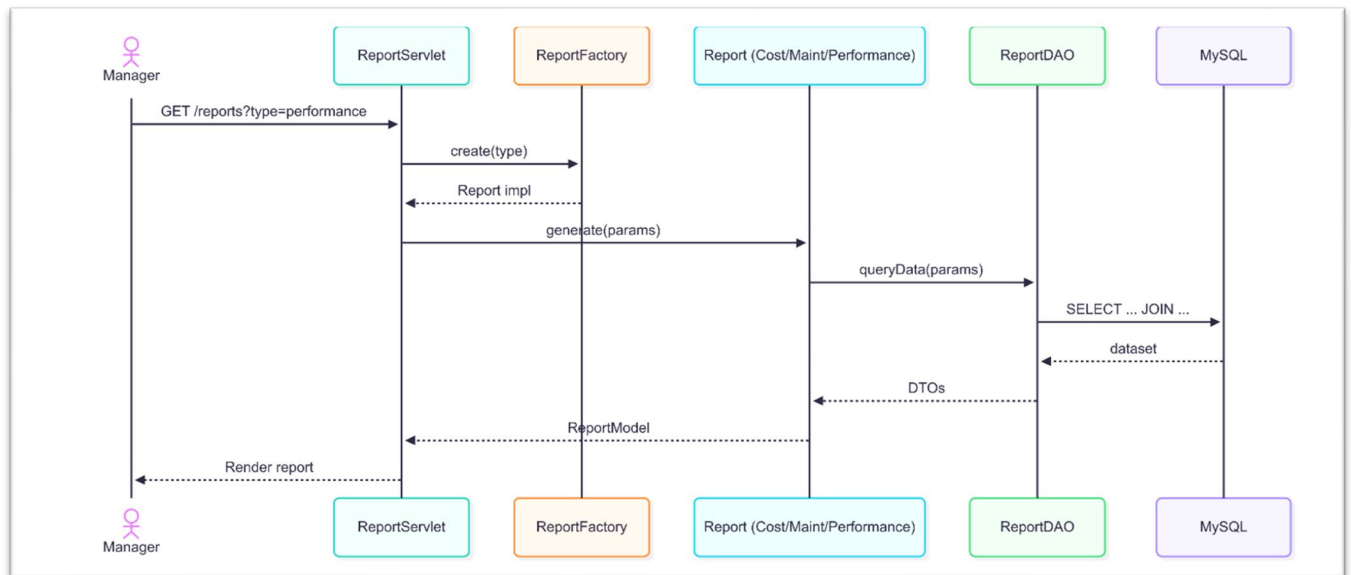


Figure 15: FR06 Generate Report Sequence Diagram

7. Data Architecture

The data architecture of the Public Transit Fleet Management System (PTFMS) ensures reliable, structured storage of operational data while maintaining referential integrity and supporting all functional requirements. The database follows a relational model implemented in MySQL 8.0, designed to align with the system's three-tier architecture and support both transactional efficiency and scalability.

7.1 Logical Data Model (ERD)

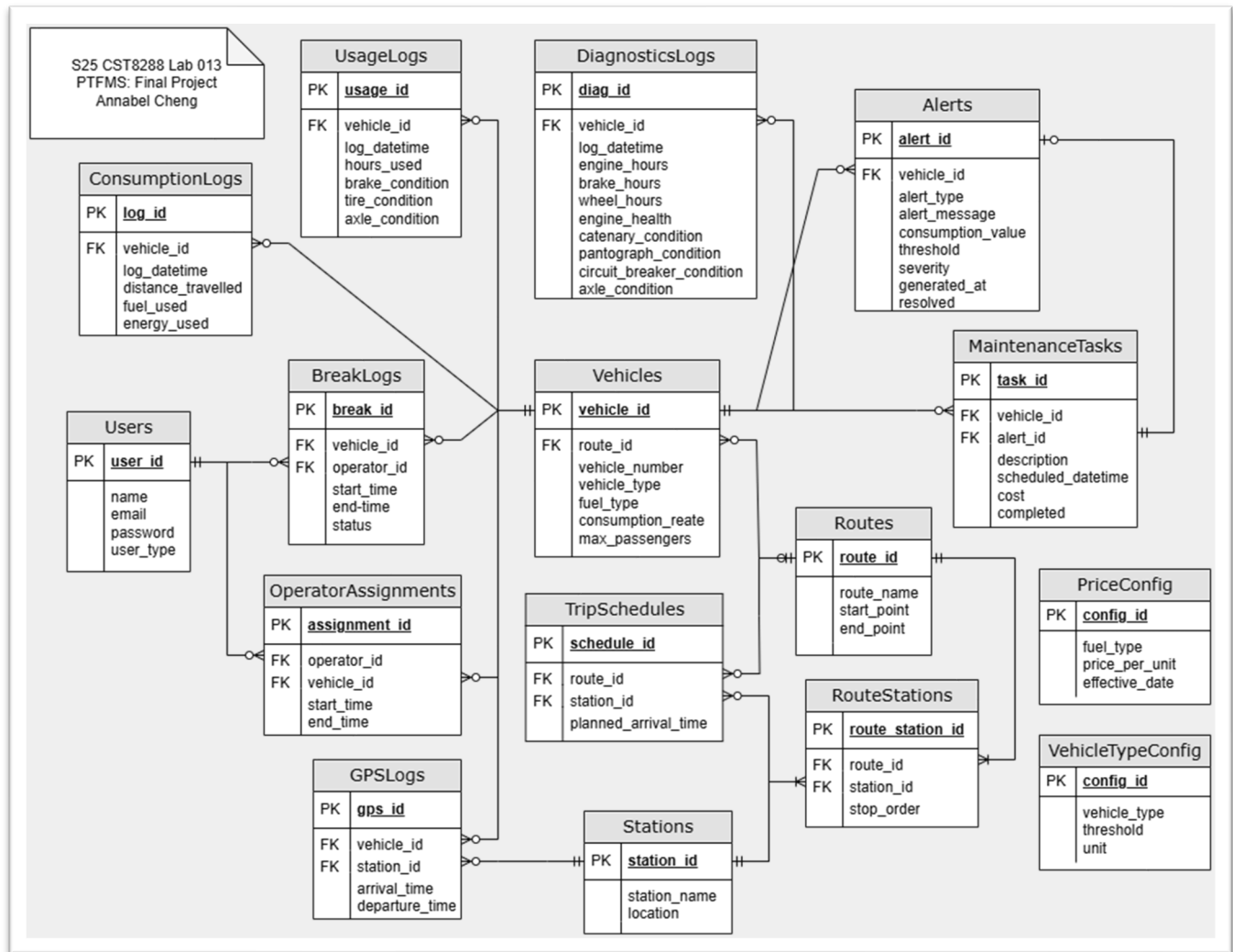


Figure 16: Logical Entity–Relationship Diagram (ERD)

The **Logical Data Model** represents the core business entities and their relationships without implementation-level details. (See Figure 2 Logical Entity–Relationship Diagram (ERD))

- **Entities** include Users, Vehicles, Routes, Stations, GPSLogs, MaintenanceTasks, UsageLogs, and DiagnosticsLogs.
- **Relationships** define business rules, such as each MaintenanceTask being associated with one Vehicle, and each GPSLog referencing both a Vehicle and a Station.
- **Normalisation** is applied to minimise redundancy and improve data consistency, with common attributes centralised into dedicated tables.

7.2 Physical Data Model (Schema Diagram)

The **Physical Data Model** specifies the MySQL implementation of the logical design, including table structures, data types, constraints, and indexing strategies. (See Figure 3 Physical Data Model (Schema Diagram))

- **Data Types:** Chosen for precision and efficiency (e.g., INT for identifiers, VARCHAR for text, DECIMAL for numeric measures, TIMESTAMP for time-based records).
- **Primary Keys:** Ensure uniqueness for each record in a table.
- **Foreign Keys:** Maintain referential integrity between related tables. All foreign keys use the default RESTRICT behaviour for ON DELETE and ON UPDATE, preventing automatic cascading changes. This design choice ensures that all deletions and updates to parent records are explicitly managed by the application layer (Servlets and DAO classes), reducing the risk of unintended data loss.
- **Indexes:** Applied to frequently queried columns (e.g., vehicle_id, route_id, scheduled_datetime) to improve query performance.

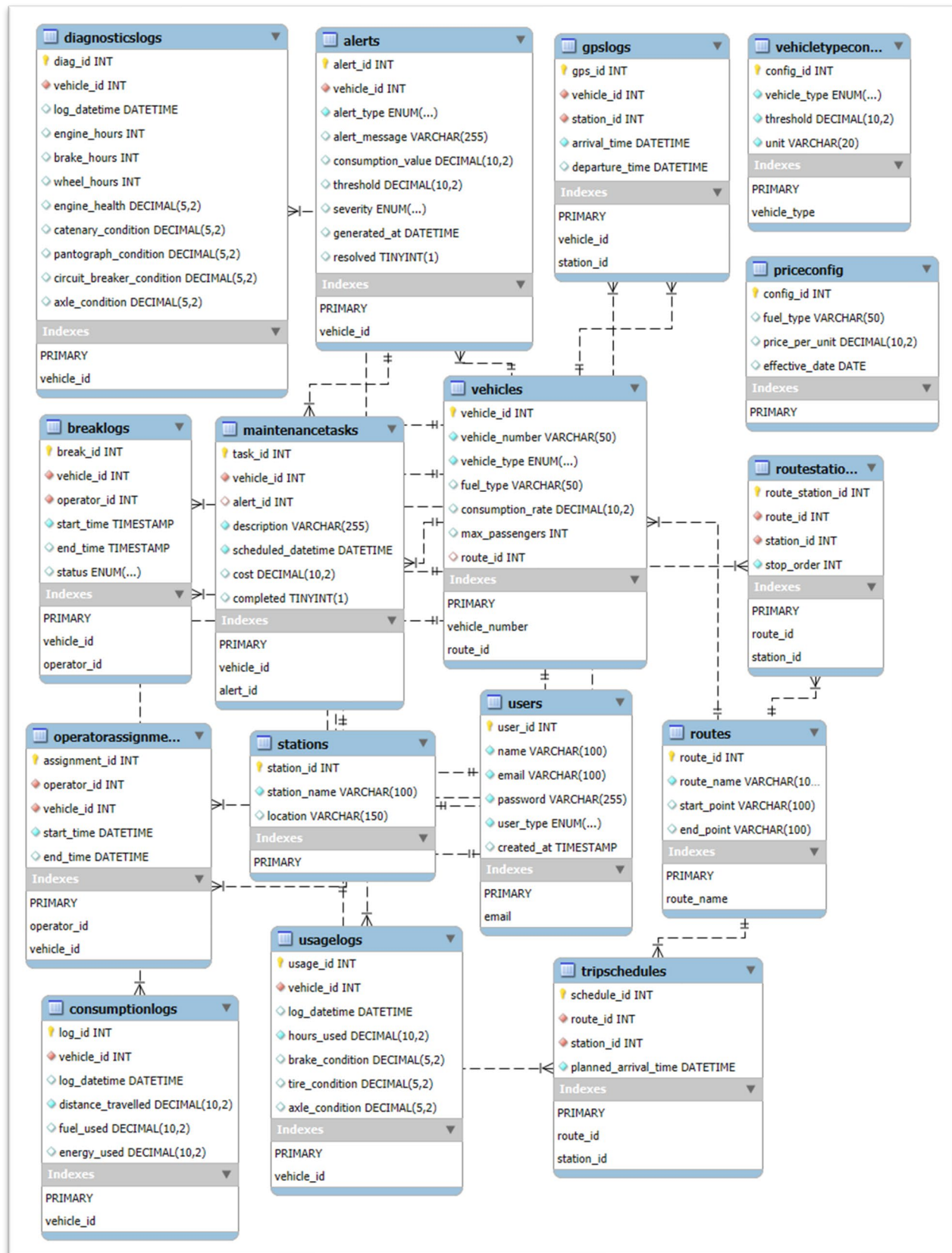


Figure 17 Physical Data Model (Schema Diagram)

7.3 Summary

By pairing the Logical ERD with the Physical Schema Diagram, the PTFMS database design demonstrates both the conceptual business model and its technical MySQL implementation. The absence of cascading delete/update rules is intentional, enforcing stricter control at the application layer to preserve data integrity. This layered design supports maintainability, scalability, and reliable system performance in line with the overall project architecture.

8. Security Architecture

The security architecture of PTFMS is designed to safeguard system integrity, protect sensitive transit data, and enforce strict role-based access across its three-tier structure. It combines robust authentication, granular access control, and secure data handling practices to prevent unauthorised use and data breaches.

8.1 Authentication & Authorization

The system implements **role-based authentication** to ensure that only authorised users can access specific functions. User credentials are stored in the Users table with unique email identifiers, and authentication is managed by LoginServlet in the Presentation Layer, delegating to UserService and UserDao for business and database logic.

- Roles **Transit Manager** and **Operator** are separated so that sensitive operations (vehicle registration, maintenance scheduling, report generation) are restricted to Managers, while Operators can log GPS and break activities.
- The AuthFilter verifies active sessions and redirects unauthenticated users to the login page.

8.2 Data Security and Session Management

Security measures also extend to session control and data handling:

- **Session Security:** A session is created at login and invalidated at logout (LogoutServlet) to prevent reuse.
- **Input Validation:** All parameters are validated in Servlets (e.g., AddVehicleServlet, UpdateMaintenanceServlet) and service classes to block injection attacks.
- **Database Protection:** Prepared statements in DAO classes (UserDao, MaintenanceDao, VehicleDao, etc.) prevent SQL injection, and credentials are stored securely in database.properties.

This layered security design ensures only authenticated, authorised, and validated requests are processed, preserving confidentiality, integrity, and availability of system resources.

9. Deployment Architecture

The PTFMS deployment uses a three-tier Jakarta EE model, built with Java 21, Tomcat 10+, and MySQL 8, managed via Maven for portability and scalability.

9.1 Infrastructure

PTFMS is deployed as a standard **Jakarta EE web application** packaged in a **WAR** and served by **Apache Tomcat 10+**. The build targets **Java 21** and uses Jakarta APIs in **provided** scope, enabling the application server to supply the servlet container at runtime while keeping the artifact minimal. Runtime integrations include:

- **Servlet stack:** jakarta.servlet-api 6.x (provided) aligned with Tomcat 10+'s Jakarta namespace.
- **Persistence/DB connectivity:** mysql-connector-j 8.x for JDBC access to a MySQL 8 database.
- **JSON processing:** gson for lightweight serialization between servlets and AJAX clients.
- **Testing toolchain:** JUnit 4 and JUnit 5 (Jupiter API/Engine/Params) scoped to test, ensuring no test libraries are shipped in the WAR.

This infrastructure maps directly to the project's three-tier design:

- **Presentation:** JSP/HTML/CSS delivered by Tomcat.
- **Business:** Java 21 servlets and services (DAO/Strategy/Builder/Observer/Command patterns) enforcing domain rules.
- **Data:** MySQL 8 with a pooled DataSource configured via externalized properties for environment portability.

9.2 Deployment Model

The deployment follows a **single-node application server + database** model for development/demo, with a clear path to scale:

- **Packaging & Build:** Maven-driven build using maven-compiler-plugin (source/target 21) and maven-war-plugin to produce a .war. CI can attach unit/integration tests with JUnit 4/5 before packaging.
- **Configuration:** database.properties externalizes JDBC URL, credentials, and pool settings; servlet context parameters are set in web.xml/Tomcat context. Secrets are not embedded in source.
- **Runtime Topology (Dev/Demo):**
 - Tomcat 10+ hosts the WAR.
 - MySQL 8 may be co-located for simplicity.

This model balances **ease of deployment** for course deliverables with **clean separation of concerns** and an upgrade path toward multi-tier or cloud-based hosting without refactoring core application code.

10. Testing Model

The PTFMS testing model combines realistic SQL datasets with targeted JUnit integration tests to verify functionality, role-based access, and data integrity across all system layers:

10.1 Functional Testing Approach

To ensure full compliance with the PTFMS functional requirements, the testing process combined realistic data simulation, role-specific verification, and layered validation at both the business and data access levels. A representative SQL dataset was prepared (ptfms-MySQL.sql) containing operationally realistic records.

This included one Manager account (tm@algonquin.com) and two Operator accounts (op@algonquin.com and op2@algonquin.com), each with the password cst8288. These accounts were used to systematically test all Manager-exclusive, Operator-exclusive, and shared functionalities throughout development.

During iterative builds, these accounts allowed verification of:

- Accurate generation of reports, consumption and predictive maintenance alerts.
- Correct functioning of all CRUD operations in vehicle, maintenance, and GPS logging workflows.
- Role-based access control and authentication processes.

10.2 Automated Unit and Integrating Testing

Automated testing was implemented using **JUnit** under the test package to support regression testing and business logic validation:

- **business.UserServiceIT:**

Integration test validating authentication logic and authorisation checks for both Manager and Operator roles. This ensured that valid credentials retrieved correct user objects with appropriate permissions, and invalid inputs produced expected null or error results.

- **dataaccess.MaintenanceDAOIT:**

Data access layer integration tests covering all scheduling-related DAO methods. This included validation of maintenance booking rules, prevention of duplicate date scheduling, and correct retrieval of task data for dashboards and reports.

By targeting both the **Business Layer** and **Data Access Layer**, these tests verified correctness across the application's 3-tier architecture, reduced manual retesting, and provided early detection of regressions when modifying related components.

10.3 Testing Summary

This combined realistic data-driven manual testing and JUnit-based automated testing approach ensured that:

- All functional requirements were met from the perspectives of both Manager and Operator roles.
- Data integrity was maintained, especially in scheduling and alert generation.
- Application behaviour remained consistent and correct across iterative development cycles.

The model provided confidence in system reliability for the final deployment and demonstration.

11. Conclusion

The Public Transit Fleet Management System (PTFMS) is an organized, maintainable, and scalable solution for managing transit operations. It includes vehicle management, GPS tracking, energy and fuel monitoring, predictive maintenance, and complete reporting. By following a three-tier architecture and employing well-established design patterns, the system provides unambiguous separation of concerns, improved testability, and ease of future expansion. The database design, which is supported by a logical ERD and a physical schema, ensures data integrity and operational efficiency for all functional requirements.

While the existing system fits the established scope and functional requirements, future enhancements could include improving real-time capabilities, integrating secure external APIs for live GPS and energy data, and implementing cloud-based deployment for scalability and robustness. Additional automation in predictive analytics, along with more advanced alerting mechanisms, would enhance maintenance planning and resource optimisation. By following these development paths, PTFMS can mature into a powerful, enterprise-grade platform capable of meeting the complex demands of current public transportation systems.

References

- [1] G. o. Four, "The Decorator Pattern: Role and Function in Structural Patterns," Gang of Four, [Online]. Available: <https://www.gofpattern.com/structural/patterns/decorator-pattern.php>. [Accessed 02 08 2025].